Fake News Detection with LSTM and Transformers

Practical Data Mining project guide | Dataset: Kaggle Fake and Real News Dataset (Fake.csv and True.csv)

Goal: build and compare a sequential LSTM classifier and a pretrained Transformer (DistilBERT) on the same held-out news articles. Add a TF-IDF + Logistic Regression baseline to assess whether model complexity provides measurable benefit.

1. Project scope and research questions

Primary question: How do LSTM and Transformer models compare in identifying fake versus real articles? Secondary questions: How do precision, recall, F1, runtime, and input length affect the comparison? Are models exploiting publisher formatting or topic cues instead of factual content?

2. Dataset and labels

Download Fake.csv and True.csv from https://www.kaggle.com/datasets/clmentbisaillon/fake-and-real-news-dataset . Each has title, text, subject, and date. Assign Fake = 0 and True = 1. Combine title and text for the primary input; exclude subject and date from the main models to reduce shortcuts. Inspect missing values, duplicates, label counts, and article lengths.

```
import pandas as pd
fake = pd.read_csv("Fake.csv").assign(label=0)
real = pd.read_csv("True.csv").assign(label=1)
df = pd.concat([fake, real], ignore_index=True)
df = df.dropna(subset=["title", "text"])
df["content"] = df["title"].str.strip() + " " + df["text"].str.strip()
```

3. Data quality and exploratory analysis

Create class-count bar charts, article-length histograms by label, subject-by-label cross-tabs, and top terms/bigrams. Identify exact duplicates and near-duplicates; inspect articles with conflicting labels. Report how many samples were excluded and why. News articles may share syndicated text or event-specific language: random splits can overstate generalization.

4. Prevent data leakage

Deduplicate before splitting. Keep near-identical articles and shared stories in the same split where feasible. Create stratified train/validation/test partitions (70/15/15). Fit vocabulary, TF-IDF vectorizer, and preprocessing statistics only on training data. Use validation for tuning and test only once for final reporting. Keep the same split for all models.

```
from sklearn.model_selection import train_test_split
train, temp = train_test_split(df, test_size=.30, stratify=df.label, random_state=42)
valid, test = train_test_split(temp, test_size=.50, stratify=temp.label, random_state=42)
```

5. Baseline: TF-IDF + Logistic Regression

Train a simple, strong baseline using scikit-learn Pipeline(TfidfVectorizer(...), LogisticRegression(...)). Tune n-gram range and minimum document frequency on validation data. Report test metrics and training time. This establishes

whether deep models offer real improvement.

6. LSTM implementation

Use light cleaning (normalize whitespace, optionally URLs and HTML); retain negations and meaningful punctuation where possible. Fit a word tokenizer only on train. Convert to sequences and pad/truncate to a fixed length such as 400-500 tokens. Start with Embedding(vocab_size, 128), LSTM(128), Dropout(0.5), Dense(1, sigmoid). Use binary cross-entropy, Adam, batch size 32-64, and early stopping on validation loss. Compare train/validation learning curves.

Suggested starting settings: vocabulary 30,000 words, max sequence length 500, embedding dimension 128, 5-10 epochs with early stopping. Record parameters, seed, hardware, and runtime. These are starting points, not guaranteed optimal settings.

7. Transformer implementation: DistilBERT

Fine-tune distilbert-base-uncased with Hugging Face Transformers and PyTorch. Use its matching AutoTokenizer on minimally modified original text. Start with max_length=512, batch size 8 or 16, learning rate 2e-5, 2-3 epochs, and validation-based checkpoint selection. A GPU is strongly recommended. DistilBERT is pretrained; note that comparing it with an LSTM trained from scratch is a comparison of complete modeling approaches, not architecture alone.

```
from transformers import AutoTokenizer, AutoModelForSequenceClassification
name = "distilbert-base-uncased"
tok = AutoTokenizer.from_pretrained(name)
model = AutoModelForSequenceClassification.from_pretrained(name, num_labels=2)
```

Check how often articles exceed 512 subword tokens. Truncation can omit relevant claims. As an optional experiment, compare first-512-token input against headline + selected article excerpts, applying the same rule consistently.

8. Evaluation protocol

Evaluate all models on identical test examples. Report accuracy, macro-F1, weighted-F1, class-specific precision/recall/F1, confusion matrices, and training/inference time. For binary decisions, specify that class 0 is Fake and class 1 is Real. If producing probabilities, add ROC-AUC or PR-AUC. Do not select a decision threshold on test data.

Metric	TF-IDF + LR	LSTM	DistilBERT
Accuracy	TBD	TBD	TBD
Macro F1	TBD	TBD	TBD
Fake recall	TBD	TBD	TBD
Real recall	TBD	TBD	TBD
Train time	TBD	TBD	TBD

9. Error analysis and bias stress tests

Inspect cases where LSTM and DistilBERT disagree; group errors by article length, subject, and linguistic features. Investigate Reuters-style source markers and other formatting artifacts. Run an ablation removing obvious source headers, then retrain/evaluate on a consistently transformed split. A substantial performance drop suggests reliance on source cues. Avoid claiming these classifiers verify facts against external evidence.

10. Reproducibility and deliverables

Recommended notebooks: 01_EDA.ipynb, 02_Preprocessing.ipynb, 03_TFIDF_Baseline.ipynb, 04_LSTM.ipynb, 05_DistilBERT.ipynb, 06_Comparison.ipynb. Save train/validation/test IDs, random seeds, package versions, model checkpoints, plots, and metrics. Include README with setup, dataset link, training commands, and limitations. Do not commit large raw datasets or checkpoints unless permitted.

11. Suggested timeline

Week 1: dataset audit, EDA, leakage-safe splits. **Week 2:** baseline and LSTM. **Week 3:** DistilBERT fine-tuning and tuning. **Week 4:** error analysis, ablation, charts, report, and presentation. Allocate extra time for GPU access and troubleshooting.

12. Report and presentation outline

Introduction and motivation; dataset provenance and EDA; preprocessing and split design; baseline and model architectures; experimental settings; results table and confusion matrices; error analysis and dataset-bias ablation; limitations; conclusion and future work. Include references to the Kaggle dataset, LSTM (Hochreiter and Schmidhuber, 1997), BERT (Devlin et al., 2019), and DistilBERT (Sanh et al., 2019).

13. Setup checklist

Use Python 3.10+ in a virtual environment or Jupyter environment with pandas, numpy, matplotlib, scikit-learn, tensorflow (for LSTM), torch, transformers, datasets, and accelerate. Confirm GPU availability before Transformer training. Keep raw files read-only and track derived artifacts separately.

Final caution: High in-dataset accuracy is not proof of reliable real-world misinformation detection. Labels and source/style differences may correlate strongly, and performance can deteriorate on new publishers, topics, and time periods.